



AUDIVA

CAHIER DES CHARGES

Ta musique, ta voix — bibliothèque locale, streaming, social & statistiques

Stack technique	Next.js 14+ (App Router), TypeScript, Tailwind CSS, Supabase
Sources externes	Jamendo API · TheAudioDB · IrcLib.net · lyrics.ovh
Version du document	2.0
Date	Septembre 2026

Sommaire

1. Présentation du projet
2. Objectifs et public cible
3. Stack technique et architecture
4. Sources de données externes
5. Modèle de données (schéma Supabase / PostgreSQL)
6. Cartographie des pages de l'application
7. Spécification détaillée des pages
 - 7.1 Authentification (Connexion / Inscription)
 - 7.2 Accueil (Home)
 - 7.3 Recherche
 - 7.4 Bibliothèque — Ma musique
 - 7.5 Page Album
 - 7.6 Page Artiste
 - 7.7 Page Playlist (+ partage)
 - 7.8 Lecteur plein écran (Now Playing)
 - 7.9 File d'attente (Queue)
 - 7.10 Paroles synchronisées (Lyrics)
 - 7.11 Découverte / Radio Jamendo
 - 7.12 Favoris (Morceaux & Playlists)
 - 7.13 Historique d'écoute
 - 7.14 Statistiques d'écoute & comparaison amis
 - 7.15 Amis (liste, demandes, profils)
 - 7.16 Messagerie / Chat entre amis
 - 7.17 Notifications
 - 7.18 Import & gestion de la bibliothèque locale
 - 7.19 Créer / éditer une playlist
 - 7.20 Paramètres & Profil
 - 7.21 Page À propos / Crédits & licences
 - 7.22 Page 404 / Erreur
8. Composant transverse : la barre de lecture persistante
9. Fonctionnalités transverses
10. Exigences non-fonctionnelles
11. Contraintes et limites connues
12. Roadmap de développement (phases)

1. Présentation du projet

Audiva est une application web de lecture musicale développée avec Next.js, combinant lecture de bibliothèque locale, streaming de musique libre de droits via Jamendo, paroles synchronisées, et désormais une couche **sociale** complète : système d'amis, messagerie, partage de playlists, notifications et statistiques d'écoute comparées entre amis. L'objectif est de reproduire l'expérience d'une plateforme de streaming moderne (type Spotify) tout en restant gratuite et bâtie sur des briques libres ou à quota gratuit généreux.

Ce document est la version 2.0 du cahier des charges. Il remplace Prisma par **Supabase** comme solution de base de données et d'authentification, et ajoute l'ensemble des fonctionnalités sociales et analytiques demandées : notifications, amis, chat, partage/favoris de playlists, et statistiques d'écoute personnelles avec comparaison entre amis.

2. Objectifs et public cible

Objectifs du produit

- Offrir une expérience de lecture musicale fluide et moderne, comparable aux standards du marché.
- Permettre l'écoute de la bibliothèque locale et d'un catalogue de streaming légal et gratuit (Jamendo).
- Fournir des paroles synchronisées en temps réel.
- Créer une dimension sociale : ajouter des amis, discuter, partager ses playlists et comparer ses habitudes d'écoute.
- Donner à l'utilisateur une vision claire de ses habitudes d'écoute via des statistiques personnelles.
- Rester responsive et utilisable aussi bien sur desktop que sur mobile.

Public cible

- Utilisateurs disposant d'une collection musicale personnelle souhaitant un lecteur moderne et social.
- Groupes d'amis souhaitant comparer leurs goûts musicaux et s'échanger des playlists.
- Auditeurs curieux de découvrir des artistes indépendants sous licence Creative Commons.

3. Stack technique et architecture

Domaine	Technologie	Rôle
Framework	Next.js 14+ (App Router)	Rendu, routing, API routes
Langage	TypeScript	Typage strict sur tout le projet
Style	Tailwind CSS	Design system utilitaire
État global	Zustand	État du lecteur, file d'attente, session
Backend & Base de données	Supabase (PostgreSQL managé)	Persistance de toutes les données de l'application
Authentification	Supabase Auth	Comptes utilisateurs, sessions, OAuth (Google...)
Temps réel	Supabase Realtime	Chat instantané, notifications live, statuts amis en ligne
Stockage fichiers	Supabase Storage	Avatars, pochettes de playlists personnalisées
Sécurité base de données	Row Level Security (RLS)	Chaque utilisateur n'accède qu'à ses propres données et celles partagées
Lecture audio	Élément <audio> natif + Web Audio API	Playback, égaliseur, visualiseur
Extraction métadonnées	music-metadata (npm)	Lecture tags ID3 / FLAC des fichiers locaux
PWA	next-pwa	Installation, cache offline partiel

Pourquoi Supabase plutôt qu'un ORM classique ?

Supabase apporte en une seule brique ce qui demanderait plusieurs outils séparés : une base PostgreSQL managée, un système d'authentification prêt à l'emploi, un moteur temps réel (indispensable pour le chat et les notifications live), et du stockage de fichiers. Cela simplifie fortement l'architecture par rapport à une stack Prisma + base auto-hébergée + service temps réel séparé.

Architecture applicative (schéma simplifié)

```
Client (React) → Store Zustand (lecteur, file d'attente, présence)
→ Supabase Client SDK (auth, requêtes DB, realtime, storage)
→ PostgreSQL (bibliothèque, playlists, amis, chat, stats, notifications)
→ API Routes proxy Next.js (/api/jamendo, /api/lyrics, /api/audiodb)
→ Jamendo API / TheAudioDB / lrclib.net / lyrics.ovh
```

4. Sources de données externes

Source	Usage	Authentification
Jamendo API	Streaming de morceaux complets sous licence Creative Commons, recherche, radios	client_id gratuit
TheAudioDB	Enrichissement métadonnées : biographies, genres, visuels artistes	Aucune (clé démo publique)

Source	Usage	Authentification
lrclib.net	Paroles synchronisées (format LRC avec timestamps)	Aucune
lyrics.ovh	Paroles brutes en fallback si lrclib ne trouve rien	Aucune

5. Modèle de données (schéma Supabase / PostgreSQL)

Toutes les tables ci-dessous vivent dans le projet Supabase, protégées par des règles Row Level Security garantissant qu'un utilisateur ne peut lire/modifier que ses propres données, celles de ses amis quand c'est pertinent (chat, stats comparées), ou les contenus publics (playlists partagées).

```
profiles (extension de auth.users)
  id uuid PK, references auth.users
  username text unique
  display_name text
  avatar_url text
  created_at timestamptz default now()

tracks
  id uuid PK
  title text
  artist text
  album text
  duration int
  source text -- 'local' | 'jamendo'
  file_path text
  jamendo_id text
  stream_url text
  cover_url text
  owner_id uuid references profiles(id)

playlists
  id uuid PK
  name text
  description text
  cover_url text
  owner_id uuid references profiles(id)
  is_public boolean default false
  share_token text unique -- utilisé pour le lien de partage
  created_at timestamptz default now()

playlist_tracks
  id uuid PK
  playlist_id uuid references playlists(id)
  track_id uuid references tracks(id)
  position int

lyrics
  id uuid PK
  track_id uuid unique references tracks(id)
  synced boolean default false
  content text
  fetched_at timestamptz default now()

liked_tracks
  id uuid PK
  user_id uuid references profiles(id)
  track_id uuid references tracks(id)
  liked_at timestamptz default now()

liked_playlists -- playlists mises en favoris (les siennes ou celles d'amis)
  id uuid PK
  user_id uuid references profiles(id)
  playlist_id uuid references playlists(id)
  liked_at timestamptz default now()
```

```
listening_history
  id uuid PK
  user_id uuid references profiles(id)
  track_id uuid references tracks(id)
  played_at timestamptz default now()
  -- agrégée côté requête pour produire les stats et tops

friendships
  id uuid PK
  requester_id uuid references profiles(id)
  addressee_id uuid references profiles(id)
  status text -- 'pending' | 'accepted' | 'blocked'
  created_at timestamptz default now()

messages
  id uuid PK
  sender_id uuid references profiles(id)
  receiver_id uuid references profiles(id)
  content text
  shared_track_id uuid references tracks(id) nullable
  shared_playlist_id uuid references playlists(id) nullable
  read boolean default false
  created_at timestamptz default now()

notifications
  id uuid PK
  user_id uuid references profiles(id) -- destinataire
  type text -- 'friend_request' | 'message' | 'playlist_shared' | 'system'
  actor_id uuid references profiles(id) -- utilisateur à l'origine
  payload jsonb
  read boolean default false
  created_at timestamptz default now()
```

Note sur les statistiques : aucune table dédiée n'est nécessaire pour les tops et comparaisons — elles sont calculées à la volée par des requêtes agrégées (GROUP BY) sur *listening_history*, éventuellement exposées via des vues SQL Supabase (views) pour simplifier les appels côté client (ex. *top_tracks_by_user*, *top_artists_by_user*).

6. Cartographie des pages de l'application

Route	Page	Accès
/login, /register	Authentification	Public
/	Accueil (Home)	Privé
/search	Recherche	Privé
/library	Bibliothèque — Ma musique	Privé
/album/[id]	Détail Album	Privé
/artist/[id]	Détail Artiste	Privé
/playlist/[id]	Détail Playlist (+ partage)	Privé / Public si partagée
/queue	File d'attente (vue mobile dédiée)	Privé
/discover	Découverte / Radios Jamendo	Privé
/favorites	Favoris (morceaux & playlists)	Privé
/history	Historique d'écoute	Privé
/stats	Statistiques d'écoute & comparaison amis	Privé
/friends	Amis (liste, demandes, profils)	Privé
/friends/[id]/chat	Messagerie / Chat avec un ami	Privé
/notifications	Notifications	Privé
/import	Import bibliothèque locale	Privé
/settings	Paramètres & Profil	Privé
/about	À propos, crédits, licences	Public
/*	Page 404	Public

Le lecteur (Now Playing) reste un composant persistant affiché sur toutes les pages privées, avec une icône de notifications et l'avatar utilisateur dans la barre de navigation supérieure.

7. Spécification détaillée des pages

7.1 Authentification (Connexion / Inscription)

Objectif : Permettre à l'utilisateur de créer un compte et de se connecter pour accéder à sa bibliothèque, ses amis et son historique.

Éléments d'interface

- Formulaire e-mail / mot de passe
- Bouton de connexion via Google (OAuth)
- Bascule Connexion ↔ Inscription
- Message d'erreur inline

Fonctionnalités

- Création de compte et connexion via Supabase Auth
- Création automatique d'une ligne profiles à l'inscription (trigger DB)
- Redirection vers / après connexion
- Validation des champs côté client et serveur

Sources / API utilisées : Supabase Auth

7.2 Accueil (Home)

Objectif : Point d'entrée principal : morceaux récents, playlists, suggestions Jamendo et activité des amis.

Éléments d'interface

- Bannière de bienvenue
- Rangées horizontales par catégorie
- Section 'Activité de tes amis' (dernières écoutes/playlists partagées)
- Cartes cliquables

Fonctionnalités

- Affichage des morceaux/playlists récents
- Fil d'activité amis basé sur listening_history + friendships
- Section 'Nouveautés Jamendo'
- Lecture rapide au clic sur une carte

Sources / API utilisées : Supabase + Jamendo API

7.3 Recherche

Objectif : Rechercher un morceau, artiste, album, playlist ou même un ami.

Éléments d'interface

- Barre de recherche avec debounce
- Filtres par type (Morceaux / Albums / Artistes / Playlists / Amis)

- Résultats groupés par catégorie

Fonctionnalités

- Recherche unifiée locale (Supabase) + Jamendo
- Recherche d'utilisateurs par username pour ajout en ami
- Historique des recherches récentes

Sources / API utilisées : Supabase + Jamendo API

7.4 Bibliothèque — Ma musique

Objectif : Vue centrale de la collection personnelle : morceaux, playlists, artistes et albums indexés.

Éléments d'interface

- Onglets : Morceaux / Albums / Artistes / Playlists
- Liste triable
- Bouton 'Importer de la musique'

Fonctionnalités

- Tri et filtrage
- Actions contextuelles (ajouter à une playlist, liker, partager, supprimer)
- Compteur de morceaux/playlists

Sources / API utilisées : Supabase

7.5 Page Album

Objectif : Afficher le détail d'un album avec la liste ordonnée de ses morceaux.

Éléments d'interface

- En-tête avec pochette, titre, artiste, année
- Bouton lecture album complet + aléatoire
- Liste numérotée des morceaux

Fonctionnalités

- Lecture de l'album en file d'attente complète
- Ajout à une playlist
- Badge 'source: Jamendo / Local'

Sources / API utilisées : Supabase / Jamendo API

7.6 Page Artiste

Objectif : Présenter un artiste : biographie, morceaux populaires, albums.

Éléments d'interface

- Bannière avec photo/illustration
- Biographie

- Section 'Morceaux populaires'
- Grille des albums

Fonctionnalités

- Récupération de la biographie via TheAudioDB
- Lecture rapide des morceaux populaires
- Fallback si absent de TheAudioDB

Sources / API utilisées : TheAudioDB + Jamendo API

7.7 Page Playlist (+ partage)

Objectif : Afficher, gérer et partager le contenu d'une playlist.

Éléments d'interface

- En-tête éditable (nom, description, cover)
- Liste des morceaux réordonnable par glisser-déposer
- Bouton 'Partager' (lien public ou envoi direct à un ami)
- Bouton like/favori sur la playlist elle-même

Fonctionnalités

- Réorganisation des morceaux (met à jour `playlist_tracks.position`)
- Génération d'un lien public via `share_token` si la playlist est rendue publique
- Partage direct dans le chat d'un ami (crée un message avec `shared_playlist_id`)
- Ajout de la playlist aux favoris (table `liked_playlists`), y compris celles des amis

Sources / API utilisées : Supabase

7.8 Lecteur plein écran (Now Playing)

Objectif : Vue immersive du morceau en cours de lecture.

Éléments d'interface

- Grande pochette centrée
- Barre de progression scrubbable
- Contrôles standards
- Bouton bascule vers les paroles
- Bouton 'Partager ce morceau à un ami'

Fonctionnalités

- Seek dans le morceau
- Bascule pochette / paroles synchronisées
- Partage rapide du morceau en cours vers le chat d'un ami

Sources / API utilisées : Store Zustand + IrcLib/lyrics.ovh

7.9 File d'attente (Queue)

Objectif : Visualiser et réorganiser les morceaux à venir.

Éléments d'interface

- Section 'En cours' / 'Prochains morceaux'
- Bouton vider la file

Fonctionnalités

- Réorganisation par glisser-déposer
- Ajout rapide depuis n'importe quelle liste

Sources / API utilisées : Store Zustand

7.10 Paroles synchronisées (Lyrics)

Objectif : Afficher les paroles du morceau en cours, surlignées ligne par ligne.

Éléments d'interface

- Défilement automatique
- Ligne active en surbrillance
- Indicateur synchronisé / texte brut

Fonctionnalités

- Requête Irclib.net en priorité, fallback lyrics.ovh
- Mise en cache en base (table lyrics)
- Clic sur une ligne pour seek

Sources / API utilisées : Irclib.net puis lyrics.ovh

7.11 Découverte / Radio Jamendo

Objectif : Explorer le catalogue Jamendo par genre, humeur ou popularité.

Éléments d'interface

- Filtres par genre/tag
- Grille de morceaux/albums
- Bouton 'Lancer une radio'

Fonctionnalités

- Appel paginé à l'API Jamendo
- Lecture en file continue
- Ajout direct à la bibliothèque personnelle

Sources / API utilisées : Jamendo API

7.12 Favoris (Morceaux & Playlists)

Objectif : Regrouper tous les morceaux et toutes les playlists (siennes ou d'amis) marqués comme favoris.

Éléments d'interface

- Onglets 'Morceaux likés' / 'Playlists likées'
- Liste triable
- Bouton lecture de tous les favoris

Fonctionnalités

- Ajout/retrait rapide via icône cœur, disponible partout dans l'app (morceaux et playlists)
- Une playlist partagée par un ami peut être ajoutée en favori sans en devenir propriétaire
- Tri par date d'ajout, titre ou artiste

Sources / API utilisées : Supabase (tables liked_tracks, liked_playlists)

7.13 Historique d'écoute

Objectif : Retracer les morceaux écoutés récemment, avec horodatage.

Éléments d'interface

- Liste chronologique groupée par jour
- Bouton 'Effacer l'historique'

Fonctionnalités

- Enregistrement automatique à chaque lecture significative (ex. après 30s)
- Réécoute rapide depuis l'historique

Sources / API utilisées : Supabase (table listening_history)

7.14 Statistiques d'écoute & comparaison amis

Objectif : Donner à l'utilisateur une vue analytique de ses habitudes d'écoute et lui permettre de se comparer à ses amis.

Éléments d'interface

- Sélecteur de période (7 jours / 30 jours / cette année / depuis toujours)
- Cartes 'Top 5 morceaux' et 'Top 5 artistes'
- Graphique du temps d'écoute total par période
- Sélecteur d'ami pour afficher une comparaison côte à côte
- Badge 'goûts communs' listant les artistes partagés avec un ami

Fonctionnalités

- Calcul des tops via requêtes agrégées sur listening_history (GROUP BY track/artist, COUNT, ORDER BY)
- Comparaison chiffrée avec un ami sélectionné (temps d'écoute, artistes en commun, morceau le plus écouté de chacun)
- Export ou partage du récapitulatif dans le chat (façon 'Wrapped')
- Respect de la vie privée : un utilisateur ne voit les stats détaillées d'un ami que si celui-ci a accepté la demande d'amitié

Sources / API utilisées : Supabase (vues agrégées sur listening_history)

7.15 Amis (liste, demandes, profils)

Objectif : Gérer son réseau social au sein de l'application : ajouter, accepter, consulter ses amis.

Éléments d'interface

- Onglets 'Mes amis' / 'Demandes reçues' / 'Demandes envoyées'
- Barre de recherche d'utilisateurs par username
- Carte profil d'ami (avatar, pseudo, top artiste du moment)
- Bouton 'Ajouter en ami' / 'Retirer' / 'Bloquer'

Fonctionnalités

- Envoi et acceptation/refus de demandes d'amitié (table friendships, status)
- Notification automatique envoyée à la réception d'une demande
- Accès rapide au profil, aux playlists publiques et aux statistiques comparées d'un ami
- Bouton direct vers le chat avec cet ami

Sources / API utilisées : Supabase (table friendships) + Realtime pour le statut en ligne

7.16 Messagerie / Chat entre amis

Objectif : Permettre une conversation privée en temps réel entre deux amis, avec partage de morceaux et playlists.

Éléments d'interface

- Liste des conversations à gauche (desktop) ou vue dédiée (mobile)
- Fil de discussion avec horodatage
- Champ de saisie avec bouton d'envoi
- Aperçu enrichi pour un morceau ou une playlist partagée (mini-carte cliquable dans le chat)

Fonctionnalités

- Envoi/réception de messages en temps réel via Supabase Realtime
- Partage d'un morceau ou d'une playlist directement dans la conversation (shared_track_id / shared_playlist_id)
- Indicateur de lecture (read/unread)
- Notification poussée à la réception d'un nouveau message si le destinataire n'est pas sur la page de chat

Sources / API utilisées : Supabase Realtime

7.17 Notifications

Objectif : Centraliser tous les événements nécessitant l'attention de l'utilisateur.

Éléments d'interface

- Icône cloche avec badge de compteur non lues, dans la barre de navigation
- Liste chronologique des notifications

- Icône distincte par type (demande d'ami, message, playlist partagée, système)
- Bouton 'Tout marquer comme lu'

Fonctionnalités

- Réception en temps réel via Supabase Realtime (nouvelle ligne dans la table notifications)
- Clic sur une notification → redirection contextuelle (vers le chat, la demande d'ami, la playlist...)
- Marquage lu/non lu individuel ou global

Sources / API utilisées : Supabase (table notifications) + Realtime

7.18 Import & gestion de la bibliothèque locale

Objectif : Permettre à l'utilisateur d'ajouter ses fichiers audio locaux à l'application.

Éléments d'interface

- Zone de glisser-déposer / sélecteur de dossier
- Barre de progression du scan
- Récapitulatif d'import

Fonctionnalités

- Sélection via File System Access API (ou upload multiple)
- Extraction des métadonnées ID3/FLAC
- Détection des doublons avant import

Sources / API utilisées : music-metadata + File System Access API

7.19 Créer / éditer une playlist

Objectif : Modale ou page dédiée pour créer ou modifier une playlist.

Éléments d'interface

- Champ nom (obligatoire)
- Champ description
- Upload de pochette personnalisée (Supabase Storage)
- Bascule 'playlist publique'

Fonctionnalités

- Validation du formulaire
- Création instantanée avec redirection
- Suppression avec confirmation

Sources / API utilisées : Supabase + Supabase Storage

7.20 Paramètres & Profil

Objectif : Gérer les informations du compte, la confidentialité et les préférences.

Éléments d'interface

- Section Profil (pseudo, avatar, e-mail, mot de passe)
- Section Confidentialité (qui peut voir mes stats / m'ajouter en ami)
- Section Préférences (thème clair/sombre, qualité audio)
- Bouton de déconnexion

Fonctionnalités

- Modification des informations de compte via Supabase Auth
- Réglages de confidentialité appliqués aux policies RLS
- Bascule thème persistée

Sources / API utilisées : Supabase Auth + Supabase Storage

7.21 À propos / Crédits & licences

Objectif : Page légale rappelant les sources de contenu et les licences applicables.

Éléments d'interface

- Présentation du projet
- Mentions des API utilisées
- Rappel des licences Creative Commons

Fonctionnalités

- Lien vers les conditions de chaque API tierce
- Affichage dynamique de la licence exacte d'un morceau Jamendo

Sources / API utilisées : Contenu statique + Jamendo API

7.22 Page 404 / Erreur

Objectif : Gérer proprement les routes inexistantes ou les erreurs applicatives.

Éléments d'interface

- Illustration / message clair
- Bouton retour à l'accueil

Fonctionnalités

- Capture des erreurs via error.tsx (Next.js App Router)
- Log des erreurs côté serveur

Sources / API utilisées : —

8. Composant transverse : la barre de lecture persistante

Affiché en permanence en bas de l'écran (desktop) ou en mini-barre extensible (mobile). Vit dans le layout racine, indépendamment des pages.

- Pochette miniature, titre et artiste du morceau en cours (clic → Now Playing).
- Contrôles : précédent, play/pause, suivant, aléatoire, répétition.
- Barre de progression seekable.
- Bouton like rapide, accès à la file d'attente, et bouton de partage rapide vers un ami.
- Persistance de la lecture lors de la navigation entre pages.

9. Fonctionnalités transverses

- Thème clair / sombre avec bascule persistée.
- Raccourcis clavier : espace (play/pause), flèches (suivant/précédent), M (mute).
- Glisser-déposer pour réordonner playlists et file d'attente.
- Responsive complet mobile / tablette / desktop.
- Progressive Web App : installation et cache offline partiel.
- Notifications média (Media Session API) pour le contrôle depuis le verrouillage d'écran.
- Notifications et messages en temps réel via Supabase Realtime, avec badge de compteur global dans la barre de navigation.
- Statut de présence (en ligne / hors ligne) des amis via Supabase Realtime Presence.

10. Exigences non-fonctionnelles

Catégorie	Exigence
Performance	Première lecture d'un morceau < 1,5s ; listes virtualisées au-delà de 200 éléments
Sécurité	Row Level Security Supabase sur toutes les tables ; clés API tierces gardées côté serveur
Confidentialité	Un utilisateur ne peut voir les statistiques détaillées ou envoyer un message qu'à un ami accepté
Accessibilité	Navigation clavier complète, contrastes WCAG AA, attributs ARIA sur les contrôles
Compatibilité	Chrome, Firefox, Edge, Safari récents ; dégradation gracieuse si File System Access API absente
Disponibilité API externes	Mise en cache locale des réponses Jamendo/TheAudioDB/lyrics

11. Contraintes et limites connues

- Le catalogue Jamendo est limité à la musique sous licence Creative Commons : pas d'artistes mainstream.
- TheAudioDB peut ne pas référencer certains artistes indépendants (fallback à prévoir).
- IrcLib.net et lyrics.ovh sont des services non officiels, sans garantie contractuelle de disponibilité.
- La lecture de fichiers FLAC n'est pas supportée nativement par Safari.
- La File System Access API n'est disponible que sur les navigateurs basés Chromium.
- Le plan gratuit Supabase impose des limites (taille de base, nombre de connexions Realtime simultanées, bande passante) à surveiller si l'application grandit.

12. Roadmap de développement (phases)

Phase	Contenu
Phase 1 — Fondations	Setup Next.js/TypeScript/Tailwind, projet Supabase (Auth + DB + RLS), layout + lecteur persistant basique
Phase 2 — Bibliothèque locale	Import de fichiers, extraction métadonnées, pages Bibliothèque / Album / Artiste
Phase 3 — Streaming Jamendo	Intégration API Jamendo, page Découverte, lecture streaming
Phase 4 — Playlists & favoris	Création/édition de playlists, favoris morceaux et playlists, partage de playlists
Phase 5 — Paroles & enrichissement	Intégration IrcLib/lyrics.ovh, affichage synchronisé, enrichissement TheAudioDB
Phase 6 — Social	Système d'amis (demandes, acceptation), messagerie temps réel, notifications
Phase 7 — Statistiques	Historique d'écoute, tops personnels, comparaison de statistiques entre amis
Phase 8 — Finitions	Responsive complet, PWA, thème sombre, raccourcis clavier, Media Session API, tests

Fin du cahier des charges v2.0 — document évolutif, à faire évoluer au fil du développement.